

LibreOffice Calc MLB Stats Add-In

Claude Code build prompt • balldontlie API • Java 8 • UNO Add-In • dependency-free .oxt

Objective

Build a free, open-source, MIT-licensed LibreOffice Calc Add-In in Java that exposes MLB data from the balldontlie API (<https://api.balldontlie.io>, MLB endpoints under /mlb/v1/) as spreadsheet functions. Package as a dependency-free .oxt.

Hard architectural constraints (do not deviate)

- **Java 8 source/target only.** No Java 9+ APIs — no `java.net.http.HttpClient`, no `var`, no records, no `List.of/Map.of`, no `String.isBlank`. Use `URLConnection` for all HTTP.
- **Zero third-party dependencies.** No JSON library — hand-roll a minimal JSON parser for the balldontlie response shape (`{"data":[...], "meta":{"..."}}` with nested objects, arrays, strings, numbers, booleans, nulls). No bundled JARs (avoids classloader conflicts in LibreOffice's embedded JVM).
- **UNO Add-In pattern** (`XAddIn / CalcAddIn`), no sidebars or dialogs. Implement `com.sun.star.sheet.XAddIn`, `com.sun.star.lang.XServiceName`, `com.sun.star.lang.XServiceInfo`, plus the custom function interface.
- **CompatibilityName set in the .xcu for every exposed function** — non-negotiable; prevents silent function-name data loss on XLS/XLSX round-trips.
- **Non-blocking cell functions.** Never block on network I/O. Use cache + background-fetch + sentinel-return: on a cache miss, start a background fetch on a worker thread, immediately return a sentinel (e.g. `#FETCHING`), and cache the result so a recalculation (`Ctrl+Shift+F9`) returns real data. Cache with a reasonable TTL.
- **Build dependency-free** using Ant or plain `javac/jar/zip` into an .oxt. Provide the build script.

API specifics (verified against the balldontlie MLB OpenAPI spec)

- **Auth:** raw Authorization header containing the API key, **no Bearer prefix**.
- **API key resolution order** (mirror the NBA add-in): (1) system property `balldontlie.api.key`, (2) env var `BALLDONTLIE_API_KEY`, (3) properties file at `~/.config/libreoffice-mlb/balldontlie.properties` (key `api.key`). Never hardcode the key. If none found, return `#NO_API_KEY`.
- **Base URL:** `https://api.balldontlie.io`; all MLB paths under `/mlb/v1/`.
- **Response envelope:** `{"data":[...], "meta":{"next_cursor", "prev_cursor", "per_page"}}`. Pagination is **cursor-based** (cursor param), not page-based. `per_page` max is 100. Follow `next_cursor` until exhausted when needed.
- **Array query params use [] suffix and repeat:** `team_ids[], player_ids[], game_ids[], seasons[], dates[]`. URL-encode values.
- **Free-tier rate limit is low (~5 req/min).** Use a generous cache TTL, coalesce duplicate in-flight requests, and handle HTTP 429 with a `#RATE_LIMIT` sentinel (no retry-storm).
- **Tier gating (important):** On the free tier only `/mlb/v1/teams`, `/mlb/v1/players`, and `/mlb/v1/games` are available. `standings`, `season_stats`, and per-game stats require paid tiers. Make free-tier functions the core; implement paid ones too but return `#TIER` on 401/403 tier responses so the add-in degrades gracefully.

Functions to expose

Core (free tier)

Function	Description
<code>MLBTEAMS([league], [division])</code>	Multi-column: id, abbreviation, display_name, location, league, division. league = "American"/"National"; division = "East"/"Central"/"West".
<code>MLBTEAM(teamId, field)</code>	Scalar lookup of one team field.
<code>MLBPLAYERSEARCH(searchText, [maxRows])</code>	Search players by name; multi-column: id, full_name, position, team abbr, active.
<code>MLBPLAYER(playerId, field)</code>	Scalar player field (full_name, position, jersey, bats_throws, dob, height, weight, team, ...).
<code>MLBGAMES(dateOrSeason, [teamId], [maxRows])</code>	Games for a date (YYYY-MM-DD → dates[]) or a season (int → seasons[]). Multi-column: id, date, away_team_name, away_runs, home_team_name, home_runs, status, season_type. Detect date vs. season.
<code>MLBGAME(gameId, field)</code>	Scalar game field, incl. nested home_team_data.runs / away_team_data.runs.

Paid tier (implement, but #TIER-degrade gracefully)

Function	Description
<code>MLBSTANDINGS(season)</code>	Multi-column: team, league, division, wins, losses, win_percent, games_behind, run_differential. Requires season (int).
<code>MLBSEASONSTATS(playerId, season, field)</code>	Scalar from /season_stats (batting_avg, batting_hr, batting_ops, pitching_era, pitching_whip, pitching_k, ...). Use documented MLBSeasonStats keys.
<code>MLBSTAT(playerId, gameId, field)</code>	Scalar from /stats for a single game/player line.

For every multi-column function, document that output spills into adjacent cells and that the user must size the range (or use a dynamic array where supported). For date/datetime fields, return ISO strings and let the user format — or return LibreOffice date serials, noting the epoch is 1899-12-30 (day 0).

Field-name reference (use these exact JSON keys)

<code>MLBTeam</code>	id, slug, abbreviation, display_name, short_display_name, name, location, league, division
<code>MLBPlayer</code>	id, first_name, last_name, full_name, position, jersey, bats_throws, dob, age, height, weight, college, draft, active, birth_place, team{...}
<code>MLBGame</code>	id, date, season, season_type, postseason, home_team_name, away_team_name, status, venue, attendance, home_team_data{runs,hits,errors,inning_scores}, away_team_data{...}
<code>MLBStandings</code>	team{...}, league_name, division_name, wins, losses, win_percent, games_behind, run_differential, runs_scored, runs_allowed, streak, last_ten_games
<code>MLBSeasonStats</code>	batting_* (batting_avg, batting_hr, batting_rbi, batting_ops, batting_obp, batting_slg, batting_war, ...); pitching_* (pitching_era, pitching_whip, pitching_k, pitching_w, pitching_l, pitching_sv, pitching_ip, pitching_war, ...); fielding_* (fielding_fp, fielding_e, ...)

Deliverables

- Full Java source tree (add-in class, function interface, JSON parser, HTTP client, cache/background-fetch manager, API-key resolver).
- CalcAddIn.xcu with CompatibilityName, display names, descriptions, and argument metadata for every function.
- description.xml, manifest.xml, and other .oxt packaging files.

- Build script (Ant `build.xml` or a plain shell script) producing the `.oxt` with no external dependencies.
- `README.md` (function reference with examples, tier caveats, rate-limit note, date-formatting note), `INSTALL.md` (API-key setup via all three methods and the `~/.config/libreoffice-mlb/` path — note macOS uses `~/Library/Application Support/`), and `LICENSE` (MIT).

Before writing code

Confirm you'll reuse the exact key-resolution, cache/sentinel, and JSON-parsing patterns from the NBA/`balldontlie` add-in (same API family, same auth-header convention), adapting only the endpoints, function names (MLB*), the config directory (`libreoffice-mlb`), and the field mappings above.